

Práctica Evaluada

Simulador Web de Ascensor con Gestión de Estados, Eventos, Asincronía y Persistencia Local

1. Presentación de la práctica

Como parte de la asignatura **Desarrollo de Aplicaciones Web**, los estudiantes deberán desarrollar una práctica evaluada orientada al dominio de la programación del lado del cliente mediante tecnologías Web.

La práctica consistirá en construir un **simulador web de ascensor**, donde se represente de forma sencilla un conjunto de pisos, se permita realizar llamadas al ascensor desde distintos niveles y se programe la lógica necesaria para atender dichas solicitudes.

El simulador deberá manejar el estado del ascensor, procesar eventos generados por el usuario, actualizar la interfaz de forma dinámica y simular el desplazamiento entre pisos mediante procesos temporizados o asíncronos.

Además, la práctica incorporará una forma básica de persistencia de datos del lado del cliente, utilizando almacenamiento local del navegador.

El propósito principal de esta práctica no es construir una interfaz visualmente compleja, sino demostrar dominio de la lógica de programación, el manejo de eventos, la actualización de la interfaz, la gestión del estado, la asincronía y la persistencia básica en una aplicación Web.

2. Propósito formativo

Esta práctica busca que los estudiantes fortalezcan habilidades fundamentales para el desarrollo de aplicaciones Web interactivas, especialmente:

- construcción de interfaces con HTML;
- presentación visual mediante CSS;
- programación del lado del cliente con JavaScript;
- manejo de variables, arreglos, objetos y funciones;
- manipulación del DOM;
- manejo de eventos;
- gestión del estado de una aplicación;
- separación entre lógica y vista;
- uso de temporizadores o funciones asíncronas;
- representación visual de cambios de estado;
- diseño de algoritmos para resolver un problema;
- almacenamiento local de datos;

- depuración y explicación del código desarrollado.

El simulador permitirá comprobar si los estudiantes comprenden cómo una interfaz Web puede responder a las acciones del usuario, modificar su estado interno, actualizarse dinámicamente y conservar información básica entre ejecuciones.

3. Modalidad de trabajo

La práctica se realizará **en parejas**.

Ambos integrantes deberán participar en el desarrollo de la solución y estar en capacidad de explicar toda la lógica de funcionamiento del simulador.

La evaluación incluirá defensa por pareja y preguntas individuales. Por tanto, cada estudiante deberá poder responder sobre:

- el estado interno del simulador;
- la estructura de datos utilizada;
- la lógica de movimiento del ascensor;
- el manejo de solicitudes;
- el manejo de eventos;
- la actualización de la interfaz;
- el uso de asincronía;
- la persistencia local de datos;
- las decisiones tomadas durante el desarrollo.

No se aceptará que un integrante desconozca el funcionamiento general de la práctica.

4. Descripción general del simulador

Los estudiantes deberán construir una aplicación Web que represente un ascensor dentro de un edificio de varios pisos.

La aplicación deberá permitir que el usuario realice llamadas al ascensor desde distintos pisos. El ascensor deberá atender las solicitudes pendientes, desplazarse de forma lógica entre pisos y actualizar la interfaz para indicar su estado actual.

El sistema deberá simular que el ascensor no se teletransporta, sino que tarda un tiempo determinado en desplazarse de un piso a otro.

La aplicación también deberá conservar información básica del funcionamiento del simulador mediante almacenamiento local del navegador. Esta información podrá incluir el historial de llamadas, el último piso donde quedó el ascensor, la cantidad de solicitudes atendidas o algún otro dato relevante definido por la pareja.

5. Representación visual mínima del simulador

La interfaz del simulador podrá ser sencilla y esquemática. No se exige diseñar gráficamente un edificio real ni realizar animaciones avanzadas.

Como mínimo, la aplicación deberá mostrar una lista de pisos y señalar claramente en cuál de ellos se encuentra el ascensor en cada momento.

Cada piso deberá tener un botón para llamar al ascensor. Al presionar un botón, la solicitud deberá registrarse y el ascensor deberá desplazarse lógicamente hacia el piso solicitado.

El movimiento podrá representarse actualizando el piso actual del ascensor cada cierto intervalo de tiempo. Por ejemplo, si el ascensor se encuentra en el piso 1 y debe ir al piso 5, la interfaz podrá mostrar progresivamente:

Piso 2 → Piso 3 → Piso 4 → Piso 5

No se requiere movimiento gráfico continuo. Bastará con actualizar la posición del ascensor en la lista de pisos.

La interfaz deberá mostrar, como mínimo:

- piso actual del ascensor;
- estado del ascensor: detenido, subiendo o bajando;
- solicitudes pendientes;
- botones de llamada por piso;
- indicación visual del piso donde está el ascensor;
- información básica almacenada o recuperada del navegador, cuando aplique.

Las animaciones, puertas, sonidos, efectos visuales, diseño responsivo avanzado o simulación gráfica más elaborada serán consideradas únicamente como bonificación.

6. Ejemplo de interfaz mínima esperada

La interfaz puede ser tan sencilla como el siguiente esquema:

```

Simulador De Ascensor
Estado: Subiendo
Piso actual: 3
Solicitudes pendientes: 5, 2
Solicitudes atendidas: 4

Último evento registrado:
Llamada realizada al piso 5 desde piso actual 1
  
```

```

-----
Piso 5      [Llamar]
-----
Piso 4      [Llamar]
-----
Piso 3      [ASCENSOR] [Llamar]
-----
Piso 2      [Llamar]
-----
Piso 1      [Llamar]
-----

```

[Ver historial] [Limpiar historial] [Reiniciar simulador]

Este ejemplo es solo una referencia. Cada pareja podrá organizar la interfaz de forma distinta, siempre que se cumplan los requerimientos mínimos y el estado del ascensor sea comprensible para el usuario.

7. Requerimientos mínimos obligatorios

La práctica deberá cumplir, como mínimo, con los siguientes requerimientos:

a. Representación de pisos

La aplicación deberá mostrar un edificio de manera esquemática, mediante una lista o conjunto de pisos.

El simulador deberá tener como mínimo cinco pisos.

Cada piso deberá estar identificado visualmente con su número o nombre.

b. Representación del ascensor

La aplicación deberá indicar claramente en qué piso se encuentra el ascensor.

No se exige una animación avanzada. Puede utilizarse una etiqueta, un ícono, un color, un borde, una clase CSS o cualquier otro recurso sencillo para señalar la ubicación actual del ascensor.

La posición visual del ascensor debe estar sincronizada con el estado interno de la aplicación.

c. Botones de llamada

Cada piso deberá contar con al menos un botón para llamar al ascensor.

Cuando el usuario presione un botón, la solicitud deberá registrarse en una estructura de datos.

El sistema deberá evitar comportamientos incoherentes, como repetir indefinidamente la misma solicitud sin control.

d. Estado del ascensor

La aplicación deberá manejar variables o estructuras que representen el estado del ascensor.

Como mínimo, deberán considerarse:

- piso actual;
- piso destino;
- dirección actual;
- estado del ascensor;
- solicitudes pendientes;
- solicitudes atendidas o historial de eventos.

El estado deberá mostrarse en pantalla para facilitar la comprensión del funcionamiento del simulador.

e. Movimiento básico

El ascensor deberá desplazarse desde su piso actual hasta el piso solicitado.

El desplazamiento deberá ocurrir de forma progresiva, simulando el paso de un piso a otro.

No se aceptará que el ascensor cambie instantáneamente de un piso a otro sin representar el proceso de movimiento.

f. Manejo de solicitudes múltiples

El simulador deberá permitir más de una solicitud.

Las solicitudes deberán almacenarse y atenderse de acuerdo con una lógica definida por la pareja.

La pareja deberá explicar cómo organiza la cola de solicitudes y qué criterio usa para decidir cuál solicitud se atiende primero.

g. Actualización de la interfaz

La interfaz deberá actualizarse cada vez que cambie el estado del ascensor.

Como mínimo, deberá mostrarse:

- piso actual;
- dirección;
- solicitudes pendientes;
- estado del ascensor: detenido, subiendo, bajando o atendiendo solicitud;
- historial o resumen de eventos almacenados, si aplica.

h. Separación entre lógica y vista

Los estudiantes deberán procurar separar la lógica del simulador de la manipulación visual.

Se evaluará que el código no dependa únicamente de cambios visuales, sino que exista una representación interna clara del estado de la aplicación.

La lógica debe poder comprenderse incluso antes de analizar los estilos visuales.

i. Persistencia local

La aplicación deberá incorporar algún mecanismo básico de persistencia local utilizando almacenamiento del navegador.

Como mínimo, la pareja deberá guardar y recuperar al menos uno de los siguientes elementos:

- último piso donde quedó el ascensor;
- historial de llamadas realizadas;
- cantidad de solicitudes atendidas;
- registro de movimientos;
- configuración del simulador;
- cantidad de pisos seleccionada;
- tiempo de desplazamiento entre pisos.

La información deberá almacenarse en una estructura clara, preferiblemente en formato JSON.

8. Persistencia de datos no relacional

La práctica incorporará persistencia de datos del lado del cliente mediante almacenamiento local del navegador.

El propósito de esta persistencia no es construir un sistema administrativo ni una base de datos compleja, sino permitir que el simulador conserve información útil entre ejecuciones.

Para el alcance básico de la práctica se recomienda utilizar **localStorage**, debido a que permite almacenar datos sencillos en formato clave-valor y recuperar información luego de recargar la página.

La pareja deberá explicar:

- qué datos decidió guardar;
- por qué esos datos son útiles;
- cómo se almacenan;
- cómo se recuperan;
- cómo se actualiza la interfaz a partir de esos datos;
- qué limitaciones tiene el almacenamiento utilizado.

El uso de **IndexedDB** podrá considerarse como bonificación por representar una alternativa más cercana a una base de datos no relacional en el navegador.

9. Niveles de desarrollo

La práctica se organizará en tres niveles. Los dos primeros niveles son obligatorios. El tercer nivel será considerado como bonificación o nivel avanzado.

Nivel 1: Simulación básica obligatoria

En este nivel, la pareja deberá lograr que el ascensor se mueva desde un piso inicial hasta un piso solicitado.

Debe incluir:

- lista visual de pisos;
- indicación del piso donde está el ascensor;
- botones de llamada;
- identificación del piso actual;
- movimiento progresivo entre pisos;
- actualización básica del estado.

Ejemplo esperado:

El ascensor inicia en el piso 1.

El usuario presiona el botón del piso 5.

El ascensor sube progresivamente del piso 1 al 5.
La interfaz muestra el cambio de piso durante el recorrido.

Nivel 2: Cola de solicitudes y persistencia obligatoria

En este nivel, la pareja deberá manejar múltiples solicitudes y guardar información básica del simulador.

Debe incluir:

- registro de varias llamadas;
- cola o lista de solicitudes pendientes;
- atención secuencial de solicitudes;
- control de solicitudes repetidas;
- bloqueo de comportamientos incoherentes mientras el ascensor se mueve;
- actualización del estado en pantalla;
- almacenamiento local de información básica;
- recuperación de la información almacenada;
- explicación del criterio de atención.

Ejemplo esperado:

El ascensor está en el piso 1.
El usuario presiona los pisos 4, 2 y 5.
El sistema registra las solicitudes.
El ascensor las atiende de acuerdo con la lógica definida por la pareja.
La interfaz muestra las solicitudes pendientes y las va eliminando a medida que son atendidas.
El sistema guarda un historial básico de las llamadas o movimientos realizados.

Nivel 3: Optimización de ruta o bonificación

En este nivel, la pareja podrá implementar una lógica más eficiente para atender solicitudes según la dirección actual del ascensor.

Una posible estrategia es que, si el ascensor va subiendo, atienda primero las solicitudes que estén en su recorrido antes de cambiar de dirección.

Ejemplo esperado:

El ascensor está en el piso 1 y se dirige al piso 10.

Mientras sube, un usuario llama el ascensor desde el piso 5.

El ascensor se detiene en el piso 5 antes de continuar hacia el piso 10.

Este comportamiento se relaciona con algoritmos de planificación de recorridos, como el conocido algoritmo SCAN utilizado en sistemas operativos para organizar recorridos sobre estructuras secuenciales.

No se exige que la pareja copie un algoritmo formal, pero sí que sea capaz de razonar, implementar y defender una estrategia de optimización.

10. Reglas mínimas del simulador

Cada pareja deberá definir algunas reglas de funcionamiento. Sin embargo, como mínimo, deberán respetarse las siguientes:

- el ascensor inicia en un piso definido;
- el edificio debe tener al menos cinco pisos;
- el ascensor solo puede estar en un piso a la vez;
- el ascensor debe tener una dirección: subiendo, bajando o detenido;
- una solicitud debe registrarse al presionar un botón;
- una solicitud atendida debe eliminarse de la lista de pendientes;
- la interfaz debe reflejar el estado real del ascensor;
- el movimiento debe tomar tiempo;
- el sistema no debe romperse si el usuario presiona varios botones rápidamente;
- la información guardada en el navegador debe poder consultarse o recuperarse;
- la pareja debe poder explicar qué ocurre en cada paso del recorrido.

11. Restricciones técnicas

La práctica deberá desarrollarse como una aplicación Web del lado del cliente.

Como mínimo, se deberán utilizar:

- HTML;
- CSS;
- JavaScript.

No se permitirá el uso de PHP para esta práctica.

No se requiere backend.

No se requiere base de datos relacional.

No se requiere autenticación.

No se requiere consumo de API externa.

La práctica deberá ejecutarse en el navegador y deberá utilizar almacenamiento local para conservar información básica del simulador.

Se permitirá el uso de **localStorage** para guardar datos sencillos en formato clave-valor, tales como configuración inicial, historial de llamadas, último piso del ascensor o registro de eventos.

El uso de **IndexedDB** será considerado como bonificación, siempre que esté correctamente integrado, documentado y pueda ser explicado durante la defensa.

La práctica podrá desarrollarse utilizando **JavaScript puro** o un **framework/librería de frontend**, según la decisión de cada pareja y previa aprobación.

Se permitirá el uso de herramientas como **React, Vue, Angular, Svelte** u otras tecnologías equivalentes, siempre que su incorporación tenga sentido para la organización del código y no sustituya la comprensión de la lógica del simulador.

El uso de un framework no agregará bonificación por sí solo. La evaluación se centrará en la claridad de la lógica, la gestión del estado, el manejo de eventos, la asincronía, la persistencia local y la capacidad de defensa técnica.

El framework podrá utilizarse para:

- organizar la interfaz en componentes;
- manejar eventos;
- actualizar la vista;
- representar cambios de estado;
- mejorar la estructura del proyecto.

Sin embargo, la lógica principal del simulador deberá ser diseñada, implementada y defendida por la pareja. Esto incluye:

- cómo se representa el estado del ascensor;
- cómo se registran las solicitudes;
- cómo se actualiza la cola de llamadas;
- cómo se determina el siguiente piso destino;
- cómo se simula el tiempo de desplazamiento entre pisos;
- cómo se actualiza la interfaz cuando cambia el estado;
- cómo se evita la repetición o inconsistencia de solicitudes;
- cómo se guarda y recupera información usando almacenamiento local.

No se aceptará una solución donde los estudiantes no puedan explicar el flujo de datos, el comportamiento del estado, el algoritmo de atención de solicitudes o la persistencia implementada.

La pareja deberá indicar en el documento explicativo qué tecnología utilizó, por qué la seleccionó y qué ventajas o dificultades encontró durante el desarrollo.

No se evaluará la cantidad de tecnología utilizada, sino la claridad de la lógica, la gestión del estado, el manejo de eventos, la actualización de la interfaz, la persistencia de datos y la capacidad de defensa de la pareja.

12. Organización sugerida del código

Se recomienda organizar el código de manera clara, separando responsabilidades.

Una posible estructura es:

- archivo HTML para la estructura de la interfaz;
- archivo CSS para la presentación visual;
- archivo JavaScript para la lógica del simulador.

El archivo JavaScript debería incluir, de forma organizada:

- definición del estado inicial;
- funciones para registrar solicitudes;
- funciones para mover el ascensor;
- funciones para actualizar la interfaz;
- funciones auxiliares para validar o reorganizar solicitudes;
- funciones para guardar información en almacenamiento local;
- funciones para recuperar información almacenada;
- manejo de eventos de los botones.

Se evaluará negativamente el código desordenado, repetitivo o difícil de explicar.

13. Uso permitido de inteligencia artificial generativa

Los estudiantes podrán utilizar herramientas de inteligencia artificial generativa como apoyo para comprender conceptos, revisar errores o explorar alternativas de solución.

La IA podrá utilizarse para:

- explicar el problema;
- sugerir estructuras de datos;
- revisar errores de JavaScript;

- explicar funciones asíncronas;
- proponer ideas de interfaz;
- ayudar a depurar comportamientos inesperados;
- explicar el funcionamiento de localStorage o IndexedDB;
- apoyar la documentación del proceso.

No se permitirá entregar código que los estudiantes no puedan explicar, modificar y defender.

En esta práctica, se recomienda no pedirle a la IA que escriba directamente todo el simulador. En su lugar, los estudiantes deberían usarla como apoyo para comprender la lógica.

Por ejemplo, se considera adecuado preguntar:

- ¿Cómo puedo representar una cola de solicitudes en JavaScript?
- ¿Cómo funciona setTimeout?
- ¿Cómo puedo separar el estado de la vista?
- ¿Cómo puedo evitar solicitudes duplicadas?
- ¿Cómo puedo guardar un historial en localStorage?
- ¿Cómo puedo recuperar datos almacenados al recargar la página?
- ¿Cómo puedo pensar una estrategia para atender pisos mientras el ascensor sube?

Cada entrega deberá incluir una breve sección denominada **Uso declarado de IA**, donde se indique:

- herramienta utilizada;
- propósito de uso;
- qué sugerencias fueron aceptadas;
- qué fue modificado por la pareja;
- qué se aprendió durante el proceso.

14. Entregables

Cada pareja deberá entregar:

1. Enlace o carpeta del proyecto.
2. Archivos HTML, CSS y JavaScript organizados.
3. Breve documento explicativo.
4. Capturas de pantalla o evidencia de funcionamiento.
5. Descripción del algoritmo utilizado.
6. Explicación de la estructura de datos empleada para las solicitudes.
7. Explicación de la persistencia local implementada.
8. Sección de uso declarado de IA, si aplica.

9. Defensa técnica del funcionamiento.

15. Contenido mínimo del documento explicativo

El documento explicativo deberá incluir:

- nombre de la práctica;
- integrantes;
- descripción general del simulador;
- reglas definidas;
- cantidad de pisos;
- estado inicial del ascensor;
- estructura de datos utilizada;
- explicación del algoritmo de movimiento;
- explicación del manejo de solicitudes;
- datos almacenados localmente;
- justificación de la persistencia utilizada;
- dificultades encontradas;
- mejoras posibles;
- uso declarado de IA, si aplica.

16. Criterios de evaluación

La práctica será evaluada considerando los siguientes aspectos:

a. Funcionamiento general

Se evaluará que el simulador funcione correctamente, que el ascensor se mueva, que atienda solicitudes y que la interfaz se actualice de manera coherente.

b. Lógica de programación

Se evaluará la claridad del algoritmo, el manejo del estado, la estructura de datos utilizada y la forma en que se resuelven las solicitudes.

c. Manejo de interfaz, eventos y actualización de vista

Se evaluará que la pareja utilice adecuadamente los eventos de interacción del usuario y actualice la interfaz de manera coherente con el estado interno del simulador.

Se considerará el manejo del DOM, la selección de elementos, la modificación de clases, textos o estructuras visuales, así como la actualización de la vista a partir de los cambios del estado.

d. Asincronía

Se evaluará que el movimiento del ascensor se simule con tiempo, utilizando temporizadores, promesas, funciones asíncronas u otra estrategia equivalente.

e. Persistencia local

Se evaluará que la aplicación guarde y recupere información básica utilizando almacenamiento local del navegador.

La pareja deberá explicar qué información guarda, cómo la guarda, cómo la recupera y qué limitaciones tiene esta forma de persistencia.

f. Organización del código

Se evaluará que el código esté ordenado, comentado cuando sea necesario y dividido en funciones comprensibles.

g. Interfaz

Se evaluará que la interfaz sea clara, usable y coherente con el funcionamiento del simulador.

No se exige una interfaz visualmente sofisticada, pero sí comprensible y funcional.

h. Defensa técnica

Se evaluará la capacidad de ambos integrantes para explicar el funcionamiento del simulador, justificar sus decisiones y responder preguntas sobre el código.

17. Bonificaciones posibles

Podrán considerarse bonificaciones si la pareja cumple correctamente con los niveles obligatorios y agrega funcionalidades adicionales funcionales y defendibles.

Algunas bonificaciones posibles son:

- implementación de una estrategia de optimización de ruta;
- botones diferenciados para subir y bajar;
- animación fluida del ascensor;
- apertura y cierre de puertas;
- sonido o señal visual al llegar a un piso;
- registro histórico detallado de eventos;
- consulta del historial desde la interfaz;

- opción para limpiar el historial;
- recuperación del último estado del ascensor al recargar la página;
- almacenamiento de configuración personalizada del simulador;
- exportación del historial en formato JSON;
- visualización ordenada del registro de movimientos;
- simulación de pasajeros;
- capacidad máxima del ascensor;
- varios ascensores;
- pausa o reinicio de la simulación;
- selección dinámica de cantidad de pisos;
- uso de IndexedDB;
- pruebas unitarias de funciones principales;
- modo oscuro;
- diseño responsivo;
- visualización gráfica de la cola de solicitudes.

La bonificación solo será considerada cuando la funcionalidad esté integrada, no afecte el comportamiento básico y pueda ser explicada durante la defensa.

18. Condiciones importantes

La práctica debe ser desarrollada progresivamente y debe poder ser ejecutada en el navegador.

Los estudiantes deberán evitar soluciones copiadas que no comprendan. Durante la defensa, el docente podrá solicitar modificaciones en vivo, explicación de funciones, revisión del estado interno, revisión de la información almacenada o simulación de nuevos escenarios.

El objetivo de la práctica no es obtener el ascensor más bonito, sino demostrar dominio de la lógica de programación necesaria para construir aplicaciones Web interactivas.

19. Producto esperado

Al finalizar la práctica, cada pareja deberá entregar un simulador web funcional de ascensor, capaz de recibir solicitudes desde distintos pisos, mover el ascensor de forma progresiva, actualizar la interfaz, guardar información básica en el navegador y explicar claramente la lógica utilizada.

Esta práctica permitirá evidenciar la comprensión de los estudiantes sobre programación del lado del cliente, manejo de eventos, asincronía, estructuras de datos, gestión del estado y persistencia local en una aplicación Web.